DATA STRUCTURE LAB

Value Added Problem

Problem 1 Given a Binary Search Tree and a range [low, high]. Find all the numbers in the BST that lie in the given range. Note: Element greater than or equal to root go to the right side. Concept: Binary Search Tree

CODE:

```
#include <stdio.h>

#include <stdlib.h>

struct Node {

   int data;

   struct Node* left;

   struct Node* right;

};

struct Node* createNode(int value) {

   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

   newNode->data = value;

   newNode->left = newNode->right = NULL;

   return newNode;

}

struct Node* insert(struct Node* root, int value) {

   if (root == NULL)

      return createNode(value);


   if (value < root->data)

      root->left = insert(root->left, value);

   else

      root->right = insert(root->right, value); // >= goes right


   return root;

}
```

```
void printInRange(struct Node* root, int low, int high) {
    if (root == NULL)
        return;

    if (root->data > low)
        printInRange(root->left, low, high);

    if (root->data >= low && root->data <= high)
        printf("%d ", root->data);

    if (root->data < high)
        printInRange(root->right, low, high);
}
int main() {
    struct Node* root = NULL;
    int n, value, low, high;

    printf("Name:Sanjyot Ingole\n");
    printf("Section: C\n");
    printf("PRN:25070521162\n");

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter node values:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }
    printf("Enter low and high: ");
    scanf("%d %d", &low, &high);

    printf("Output:\n");
```

```
    printInRange(root, low, high)


    return 0;

}
```

Run

Output

```
Name:Sanjyot Ingole
Section: C
PRN:25070521162
Enter number of nodes: 5
Enter node values:
17
4
18
2
9
Enter low and high: 2
18
Output:
2 4 9 17 18

=== Code Execution Successful ===
```

Problem 2 Given a string s, composed of different combinations of '(' , ')', '{', '}', '[', ']'. Determine whether the Expression is balanced or not. An expression is balanced if: 1. Each opening bracket has a corresponding closing bracket of the same type. 2. Opening brackets must be closed in the correct order. Concept: Stack

CODE:

```
#include <stdio.h>

#include <string.h>

#include <stdbool.h>


#define MAX 100000


char stack[MAX];

int top = -1;


// Push operation

void push(char c) {

   stack[++top] = c;

}


// Pop operation

char pop() {

   if (top == -1)

      return '\0';

   return stack[top--];

}


// Check if matching pair

bool isMatchingPair(char open, char close) {

   if (open == '(' && close == ')') return true;

   if (open == '{' && close == '}') return true;

   if (open == '[' && close == ']') return true;

   return false;

}
```

```
// Function to check balanced expression
bool isBalanced(char s[]) {
    for (int i = 0; i < strlen(s); i++) {
        char ch = s[i];

        // Opening brackets
        if (ch == '(' || ch == '{' || ch == '[') {
            push(ch);
        }
        // Closing brackets
        else {
            if (top == -1)
                return false;

            char popped = pop();
            if (!isMatchingPair(popped, ch))
                return false;
        }
    }

    return (top == -1);
}

int main() {
    char s[MAX];

    printf("Name:Sanjyot Ingole\n");
    printf("Section: C\n");
    printf("PRN:25070521162\n");

    printf("Enter expression: ");
    scanf("%s", s);
```

```
    if (isBalanced(s))
        printf("true\n");
    else
        printf("false\n");

    return 0;
}
```

Run

Output

```
Name:Sanjyot Ingole
Section: C
PRN:25070521162
Enter expression: ({}[])
true


=== Code Execution Successful ===
```